DESIGN DOCUMENT | 5 NOVEMBER 2026

SmartPOS

Complete Application Design

Application design, database design, validation, business logic, the AI assistant, and how the demand forecasting model is built, trained and used.

Document	
Stack	Angular 22, ASP.NET Core 10, PostgreSQL 16, Python 3.12 (FastAPI, LightGBM), Ollama qwen3.5:9b
Scope	The whole application: POS, products, inventory, customers and loyalty, dashboard and reports, AI assistant, demand forecasting (beta)
State	As built and tested on 5 November 2026

Contents

1 System architecture 3

1.1 How a request is handled . . . 3

1.2 Security controls . . . 4

2 Application design: backend 5

2.1 Controllers and routes . . . 5

2.2 Completing a sale . . . 5

2.3 Other business rules . . . 7

3 Application design: frontend 7

3.1 Point of sale . . . 8

3.2 Forecasting page . . . 8

4 Database design 9

4.1 Tables . . . 9

4.2 Multi-shop isolation . . . 10

4.3 Stock ledger . . . 10

4.4 Forecasting schema . . . 11

4.5 Migrations . . . 11

5 Validation 11

5.1 Field rules (DataAnnotations) . . . 12

5.2 Business rules . . . 12

6 Feature flows and business logic 13

6.1 Accounts . . . 13

6.2 Products and inventory . . . 13

6.3 Dashboard, reports and insights . . . 13

6.4 Development data tools . . . 14

7 AI assistant 14

7.1 Store tools . . . 14

7.2 The grounding check . . . 15

7.3 Why the strict prompt . . . 15

8 Demand forecasting (beta) 16

8.1 Turning sales into daily series . . . 16

8.2 Features . . . 16

8.3 Training . . . 17

8.4 Accuracy on the real shops . . . 17

8.5 Making the 7-day forecast . . . 18

8.6 Stockout risk . . . 18

8.7 Reorder suggestions . . . 19

8.8 Current runs . . . 20
8.9 The synthetic benchmark: why and how . . . 20
8.10 Benchmark results . . . 21
8.11 Why stockout days are left out . . . 22
8.12 Operations and known limits . . . 22
9 API reference **23**
10 Testing **24**

1 System architecture

SmartPOS is a point-of-sale and inventory system for small shops. Any number of shops share one PostgreSQL database; each shop has exactly one owner account. Three server processes work together, and the browser only ever talks to one of them: the ASP.NET Core API.

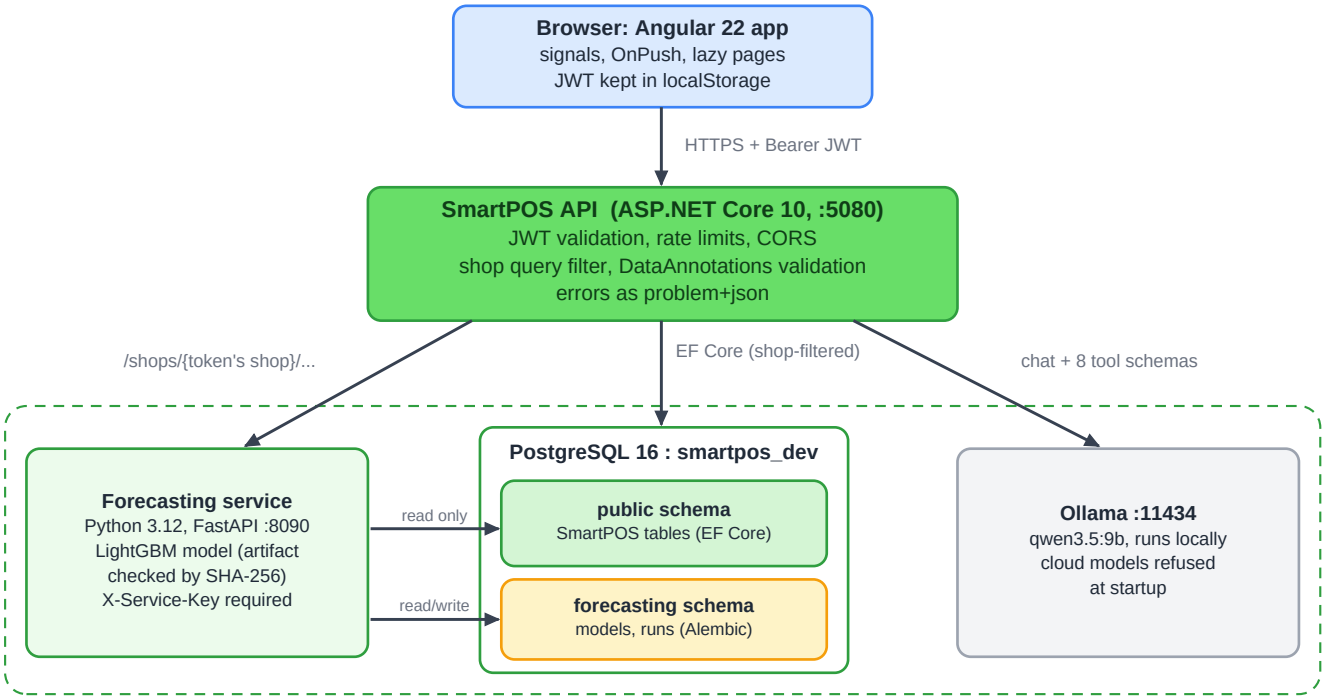
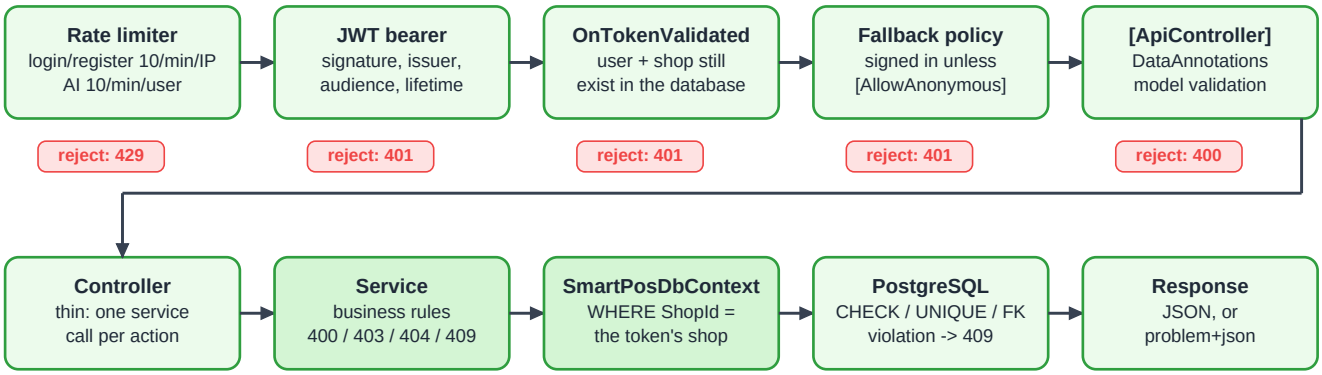


Figure 1. Services and how they connect. Dashed area: server side; only the API is reachable from the browser, the others listen on loopback.

Process	Job	Listens on	Trusts
SmartPOS API	Every browser request: auth, sales, stock, reports; proxy for AI and forecasting	:5080	a valid JWT
Forecasting service	Reads sales and stock, trains the model, writes forecast runs	127.0.0.1:8090	X-Service-Key header
Ollama	Runs the local language model for the assistant	127.0.0.1:11434	loopback only
PostgreSQL 16	public schema (EF Core) + forecasting schema (Alembic)	:5432	role smartpos

1.1 How a request is handled

Every API request passes the same gates in order. Each gate can end the request with a specific status code, so the caller always learns which rule it broke.



Every request passes these gates in order; the first one that fails ends the request.

Figure 2. Request pipeline from the network to the database and back.

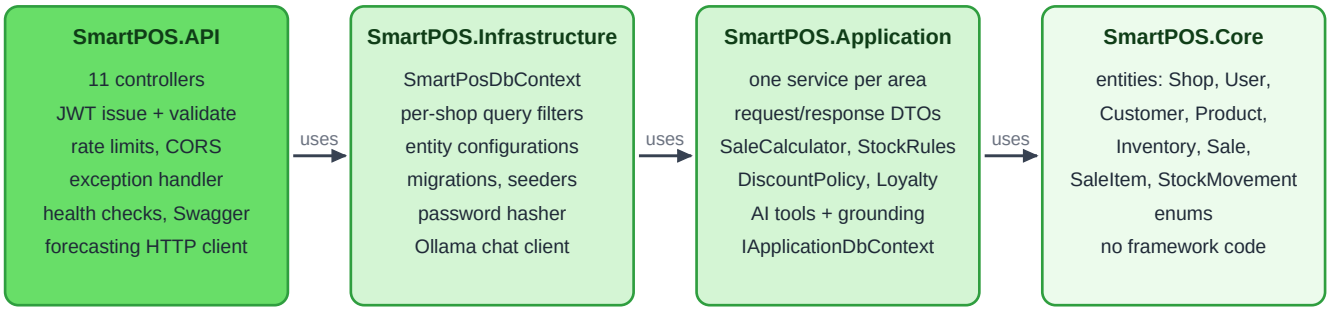
> **Tenant isolation.** The shop never comes from the browser. It is a claim inside the signed JWT, it is re-checked against the database on every request, and the DbContext applies it to every query. A forgotten *WHERE ShopId* cannot leak data, because the filter is added automatically and matches nothing when no shop is known.

1.2 Security controls

Concern	Control
Passwords	ASP.NET Core Identity PasswordHasher (PBKDF2), salted per password, rehashed when settings change
Tokens	HMAC-signed JWT, 480 minutes, claims sub/name/email/shop; issuer, audience and lifetime checked; user and shop re-checked in the database
Authorization	Fallback policy: signed in everywhere unless [AllowAnonymous] (login, register, verify-email). No roles: one owner per shop
Rate limits	login/register/verify 10/min per IP; AI 10/min per user plus one in flight; forecast generate 3/min per shop
Secrets	user-secrets in development, environment variables in production; never in appsettings; startup fails if the signing key is missing
Errors	problem+json; 500s carry only a trace id; health output has no exception text
Local AI	Ollama on loopback only; models tagged :cloud are refused; prompts never contain secrets

2 Application design: backend

The backend is one ASP.NET Core 10 solution with four projects. Dependencies point in one direction, so business rules in Application can be unit-tested without a web server or a database.



Dependencies point one way: controllers never touch EF Core types they don't need; Core depends on nothing.

Figure 3. Projects and what each one contains.

Why no repositories. Services use a small *IApplicationDbContext* interface instead of a repository per entity. A generic repository would wrap EF Core's LINQ with a weaker copy of itself. The interface adds one operation that needs care, *TryChangeStockAsync* (the atomic conditional stock update), so concurrency-sensitive SQL lives in one place.

2.1 Controllers and routes

Controller	Routes (prefix /api)	Notes
Auth	POST auth/register, auth/login, auth/verify-email, auth/resend-verification; GET auth/me	register and login return a JWT; rate limited per IP
Products	GET products, products/{id}; POST products; PUT products/{id}; DELETE products/{id}	paged search; delete refused (409) once the product has sales: deactivate instead
Categories	GET categories	distinct category text from the shop's products
Inventory	GET inventory; POST inventory/adjustments; GET inventory/movements	stock changes only through sales and adjustments
Customers	GET customers, customers/{id}; POST customers; PUT customers/{id}	phone unique per shop; points shown on the customer
Sales	POST sales; GET sales (from, to, search, paging); GET sales/{id}	the server prices and totals every bill
Dashboard	GET dashboard/summary, dashboard/insights?days=	today's figures; analytics vs the previous period
Reports	GET reports/daily-sales, reports/top-products	grouped by store-local day
Settings	GET settings/pos	max discount, currency, loyalty rule for the till
AI	POST ai/chat; GET ai/status	local model with read-only tools
Forecasting	GET forecasting/status, forecast, categories, stockout-alerts, replenishment, products/{id}/forecast, metrics; POST forecasting/generate	proxy: the path to the Python service is built from the token's shop

2.2 Completing a sale

A sale is the most important write in the system. It must never oversell, never trust a price from the browser, and leave a consistent bill, stock ledger and loyalty balance, or nothing at all.

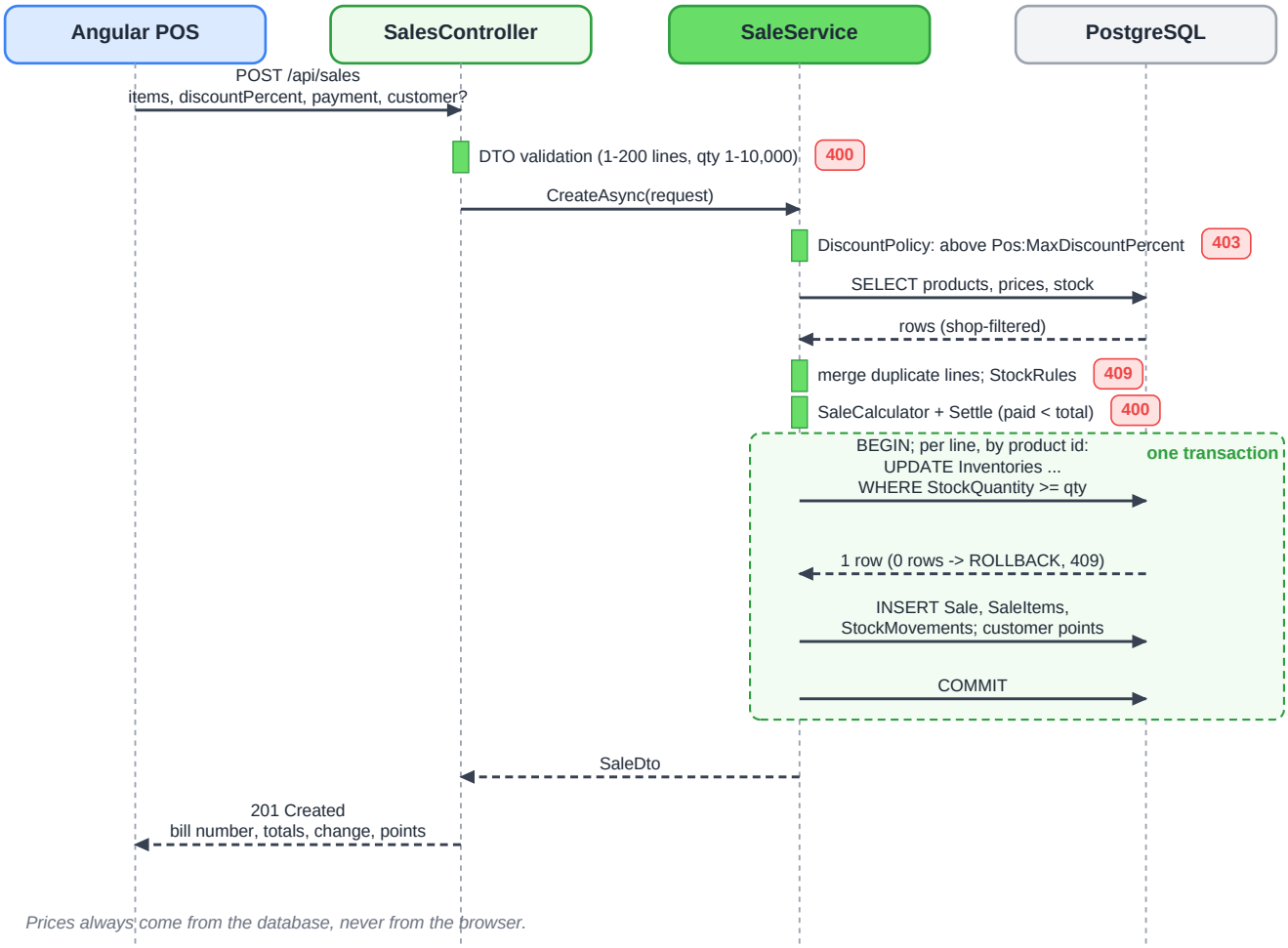


Figure 4. Sequence of a sale, with the status code each check can return.

Bill arithmetic (SaleCalculator)

SaleCalculator is the single source of truth for money. The Angular cart mirrors it in integer cents for the live preview, and both sides are tested with the same example. A worked example:

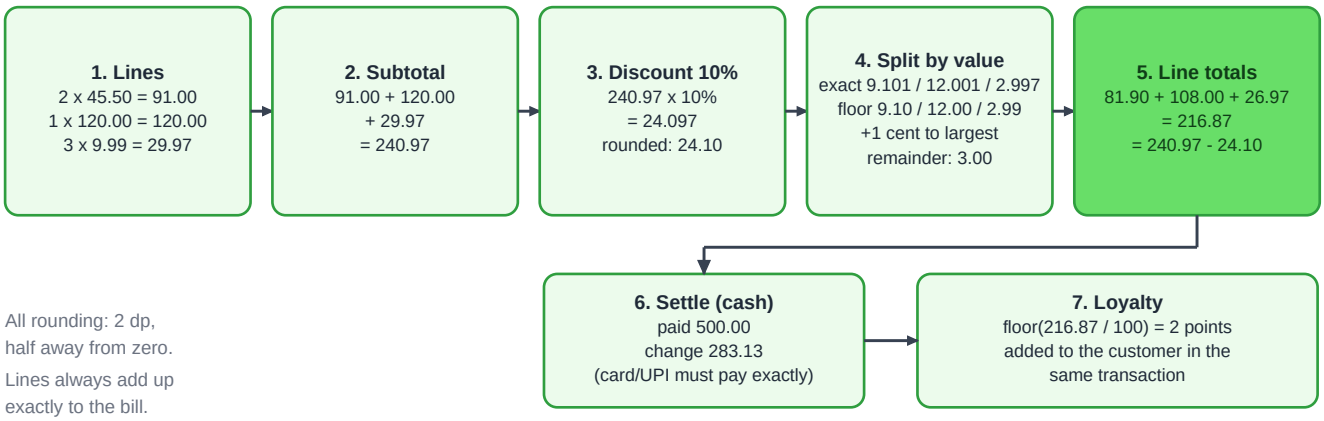


Figure 5. A 3-line bill with a 10% discount, paid in cash, for a customer.

No overselling, even under load

The pre-check gives a friendly message, but the real guard is the conditional UPDATE inside the transaction. PostgreSQL holds the row lock until commit and re-checks the WHERE clause after waiting, so concurrent checkouts for the same product are serialised. Lines are locked in product-id order so two multi-item sales cannot deadlock. A CHECK (StockQuantity >= 0) is the final safety net.

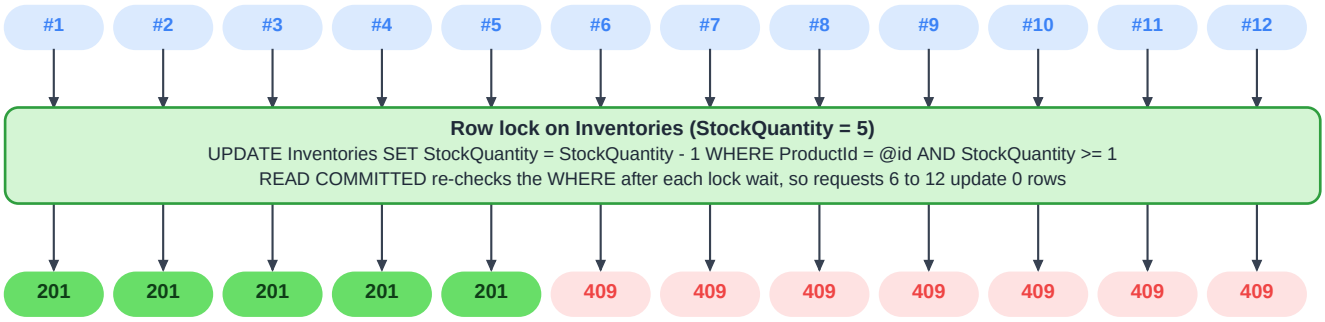


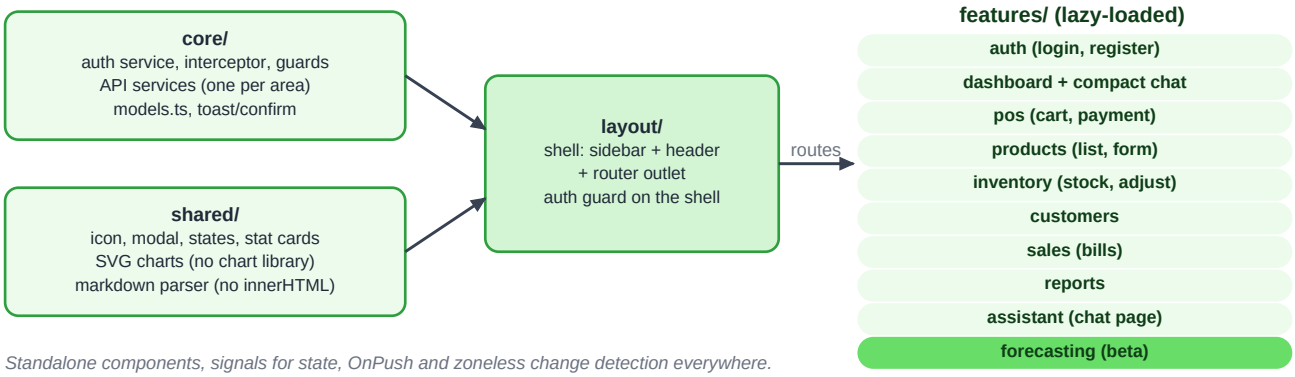
Figure 6. Measured on PostgreSQL 16: 12 concurrent one-unit sales against a stock of 5. Exactly 5 succeeded.

2.3 Other business rules

- **Discount**: one sale-level percentage, 0 to *Pos:MaxDiscountPercent* (403 above it), split across lines by value so the lines add up to the bill.
- **Payment**: cash may be over-tendered (change is returned); card, UPI and other must be paid exactly. No tax is charged.
- **Snapshots**: each SaleItem stores the product name, SKU, unit price and unit cost at the time of sale, so old bills and profit reports never change when a product is edited.
- **Loyalty**: points = floor(bill total / *Loyalty:SpendPerPoint*, default 100), stored on the sale and added to the customer in the same transaction. Redemption is not built yet.
- **Email verification**: sign-up refuses disposable domains and domains with no MX record (free DNS-over-HTTPS lookup, fails open), then emails a single-use link valid for 24 hours. Only its SHA-256 hash is stored. Not yet required for any feature.
- **Time**: timestamps are stored in UTC; 'today', date filters and daily reports use the store time zone (*Store:TimeZoneId*, Asia/Kolkata for these shops).

3 Application design: frontend

The web app is Angular 22 with standalone components. Each area of the app is a lazily loaded feature folder; shared building blocks live in *core* and *shared*.

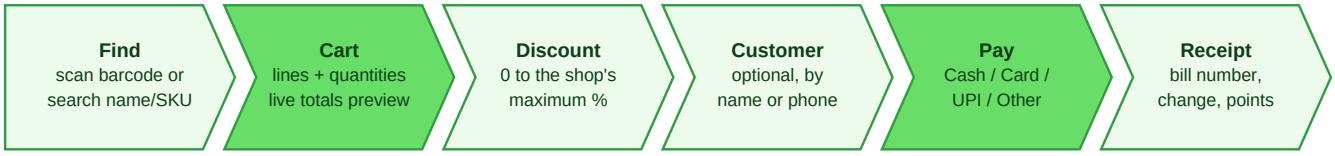


Standalone components, signals for state, OnPush and zoneless change detection everywhere.

Figure 7. Frontend structure.

Pattern	How it is used
Signals + computed	Page state is signals; derived values (filtered rows, totals, counts) are computed
Reactive forms	Validators mirror the API's rules; the API's own message is shown if it still rejects
Interceptor + guards	The interceptor adds the Bearer token; guards only shape navigation, the API enforces every rule itself
Design tokens	Colours, radii and spacing in styles.css; shared classes .card, .table, .btn, .badge
Charts	Hand-written SVG components (area, horizontal bar); no chart library
Safe Markdown	Assistant answers are parsed into data and rendered with text bindings, never innerHTML

3.1 Point of sale



The browser preview mirrors SaleCalculator; the server re-prices and recalculates on submit.

Figure 8. The till, from finding a product to the receipt.

3.2 Forecasting page

One card with four tabs (Demand, Products, Stockout alerts, Orders) keeps the page within one screen. The filters (category, product search, day) apply to every tab, tab labels show live counts, and long tables scroll inside the card with sticky headers. Arrow keys, Home and End move between tabs (WAI-ARIA tabs pattern). There is no shop selector: it is always the signed-in owner's shop.

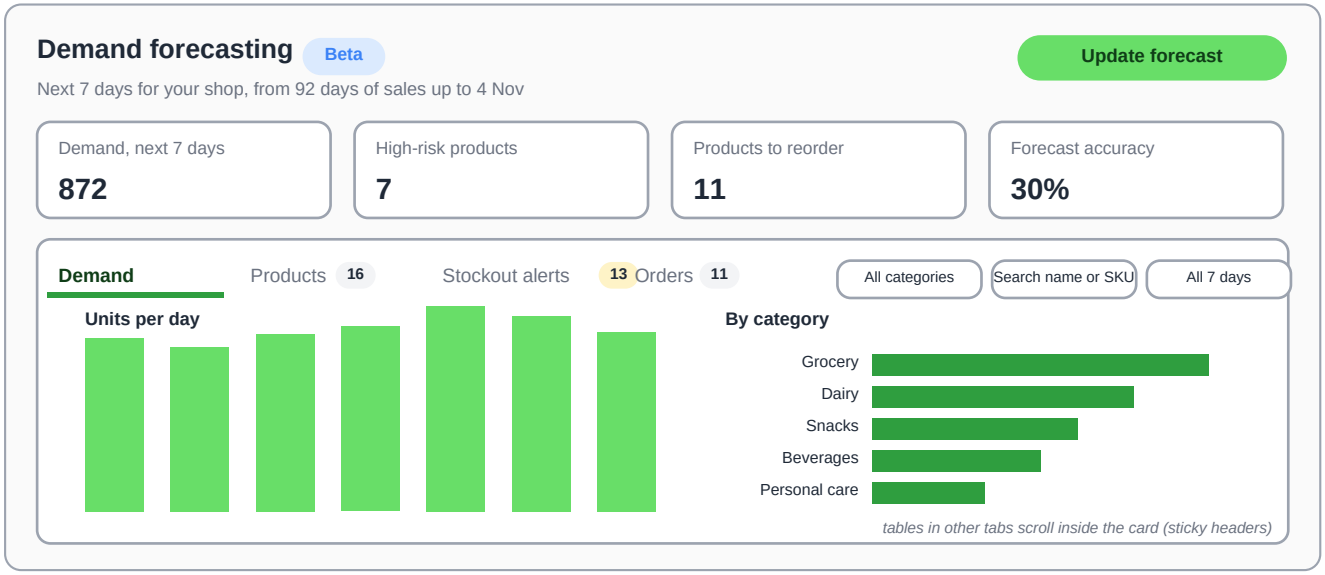


Figure 9. Wireframe of the forecasting page (values from shop 1's run).

4 Database design

One PostgreSQL 16 database, *smartpos_dev*, with two schemas: **public** holds the SmartPOS tables and is changed only by EF Core migrations; **forecasting** holds models and forecast runs and is changed only by Alembic migrations. Money is numeric with 2 decimals, timestamps are timestamptz in UTC.

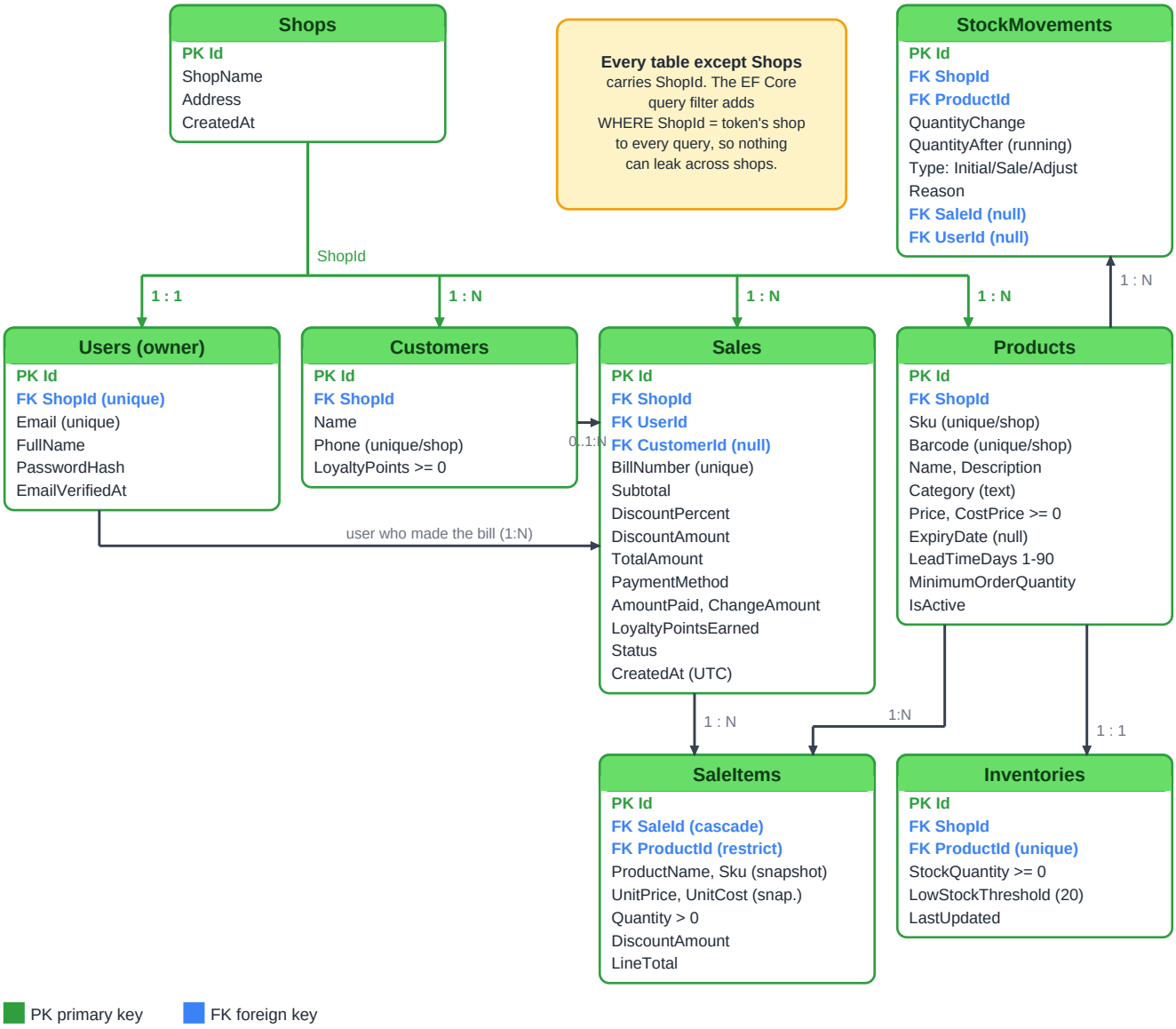
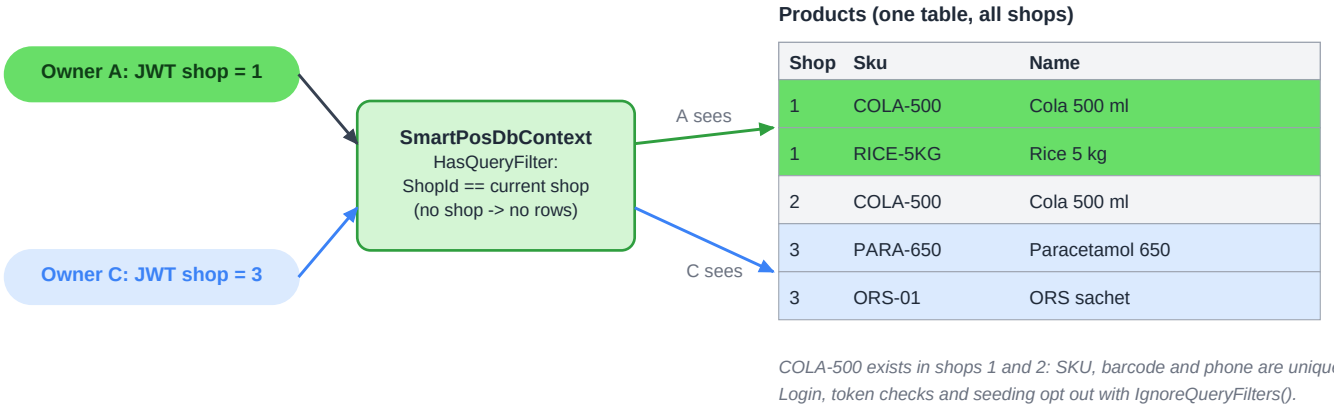


Figure 10. Entity-relationship diagram of the public schema.

4.1 Tables

Table	Purpose	Rules
Shops	a tenant (store)	root table, no ShopId
Users	the login; exactly one per shop (the owner)	Email unique across all shops, stored lower-case; ShopId unique (1:1); verification link stored as hash + expiry
Customers	people the shop sells to	unique (ShopId, Phone); LoyaltyPoints >= 0
Products	catalogue	unique (ShopId, Sku) and (ShopId, Barcode); Price, CostPrice >= 0; LeadTimeDays 1-90 (default 3); MinimumOrderQuantity >= 1 (default 1)
Inventories	stock on hand, one row per product	ProductId unique; StockQuantity >= 0; LowStockThreshold default 20
Sales	one bill per checkout	BillNumber unique (BILL-yyyymmdd-random hex); Status Pending/Completed/Cancelled; index (ShopId, CreatedAt)
SaleItems	lines of a bill	Quantity > 0; snapshots; FK to Sales cascades, FK to Products restricts (history is kept)
StockMovements	append-only stock ledger	Type InitialStock/Sale/Adjustment, QuantityAfter (running balance), Reason, optional sale and user

4.2 Multi-shop isolation



COLA-500 exists in shops 1 and 2: SKU, barcode and phone are unique per shop. Login, token checks and seeding opt out with IgnoreQueryFilters().

Figure 11. One table, many shops: the query filter decides which rows a request can see.

4.3 Stock ledger

Stock is never edited directly. A sale or an adjustment changes *Inventories.StockQuantity* and writes a *StockMovement* with the change and the resulting balance, so every unit is accounted for and the forecasting service can rebuild each day's closing stock.

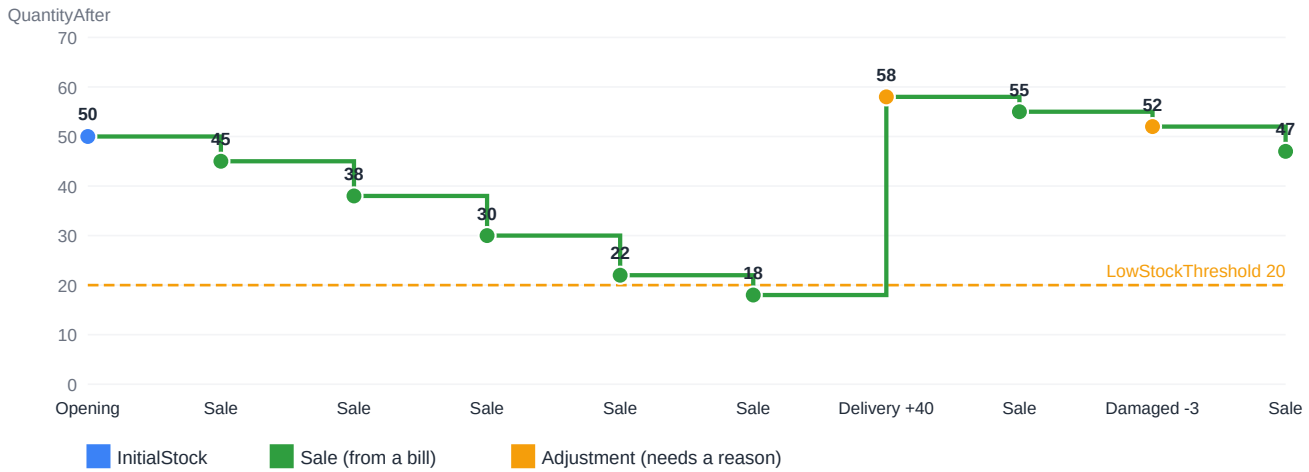


Figure 12. Example ledger for one product (illustrative values). The drop to 18 makes it 'Low'.

4.4 Forecasting schema

Table	Key	Holds
model_metadata	model_version	type, dates, features, parameters, artifact path + SHA-256, data_source (smartpos / synthetic); one active model per source (partial unique index)
model_evaluation_metrics	id	MAE, RMSE, WAPE, bias, n per model, evaluation, method and scope (overall / shop / category / product)
shop_forecast_runs	run_id (uuid)	shop_id, model version, as-of date, horizon, trading days used, generated_at
shop_demand_forecasts	run, product, date	predicted demand and the method used
shop_stockout_predictions	run, product	stock, 7-day demand, cover, stockout date, risk, lead time, safety stock, reason
shop_replenishment_recommendations	run, product	quantity, reorder point, order-up-to, MOQ, order-by date, reason

Design choice. Results hang off a run and a run belongs to one shop, so a shop's forecasts can only be reached through its own runs. shop_id and product_id are SmartPOS ids without a cross-schema foreign key, so EF Core and Alembic stay independent.

4.5 Migrations

- **EF Core** (public): InitialCreate, AddLoyaltyAndEmailVerification, RemoveTax (recalculated old bills without tax), AddProductLeadTimeAndMoq. *database/schema.sql* is a generated copy for review.
- **Alembic** (forecasting): 0001 (models and benchmark tables), 0002 (shop_* tables, data_source, wider scope_id).
- A database backup is taken before every migration that touches real data.
- PostgreSQL folds unquoted names to lower case, so column names in CHECK constraints are quoted.

5 Validation

Every input is checked at four layers. The browser check is a convenience; the server layers are the ones that count, and each returns a clear status code and message.

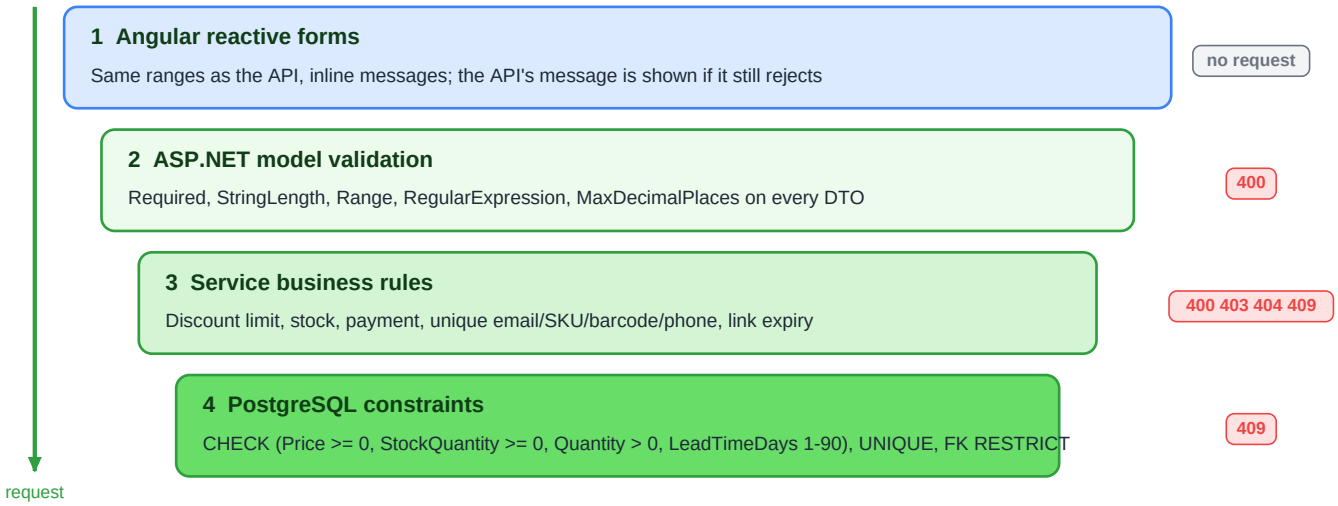


Figure 13. Validation layers and what each one rejects.

5.1 Field rules (DataAnnotations)

Request	Field	Rule
Register	full name / shop name / address	2-100 / 2-150 / up to 250 characters
Register	email	valid address, up to 254 characters
Register	password	8-128 characters with at least one letter and one digit
Product	SKU	2-50; letters, digits, '.', '_', '-'; starts with a letter or digit
Product	barcode	digits only, up to 50
Product	name / category / description	2-150 / 2-100 / up to 1000
Product	price, cost price	0 to 9,999,999.99, at most 2 decimals
Product	opening stock, low-stock threshold	0 to 1,000,000
Product	lead time / minimum order	1-90 days / 1-10,000
Customer	name / phone	2-100 / 7-20 of digits, spaces, '+', '-', '(', ')'
Sale	items	1-200 lines; product id >= 1; quantity 1-10,000
Sale	discount / amount paid	0-100% / 0 to 9,999,999,999.99, at most 2 decimals
Adjustment	change / reason	-100,000 to 100,000 (not 0) / 3-250 characters
AI chat	message / history	1-1000 characters / at most 40 turns, role user or assistant, each up to 4000

5.2 Business rules

Rule	Checked in	Result
Email already registered	AuthService	409
Disposable or undeliverable email domain	AuthService	400
Wrong email or password (same message for both)	AuthService	401
Discount above the shop's maximum	DiscountPolicy	403
Not enough stock (pre-check, or 0 rows updated)	StockRules / TryChangeStockAsync	409
Amount paid below the total; change on a non-cash payment	SaleCalculator.Settle	400
Adjustment would make stock negative, or is zero	StockRules	409 / 400
Delete a product that has sales	ProductService	409
Duplicate SKU, barcode or customer phone in the shop	service check + unique index	409
Another shop's product, customer or bill	query filter (row not visible)	404
Forecast with under 7 trading days / no model	forecasting service	409 / 503
AI question while one is running, or over 10/min	AIConcurrencyGate / rate limiter	429

6 Feature flows and business logic

6.1 Accounts

There is one kind of account. Registering creates the user and the shop they own in one transaction and signs them in. There are no seeded users and no default passwords anywhere.

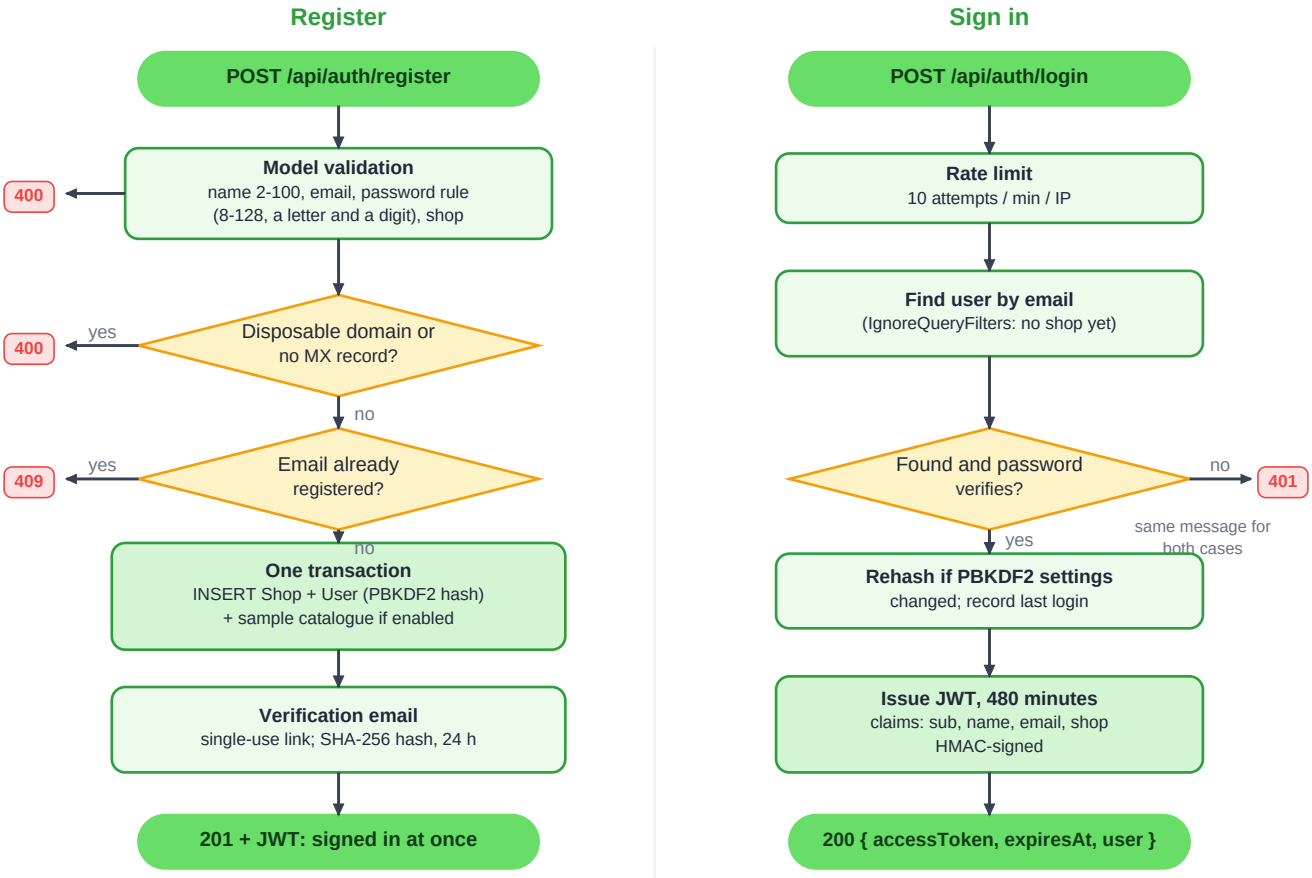


Figure 14. Registration and sign-in.

6.2 Products and inventory

- **Create**: checks SKU and barcode are unused in the shop, inserts the product, its Inventories row and an InitialStock movement for the opening stock.
- **Edit**: never changes stock (the update request has no stock field). Lead time and minimum order are used by forecasting.
- **Delete**: refused once the product appears on a bill; it is deactivated instead and stays in reports and forecasting history.
- **Adjust**: positive for a delivery, negative for damage or a count correction; a reason is required and the result cannot go below zero.
- **Status**: Out of stock at 0, Low at or below the threshold, otherwise In stock. Expiry is per product, not per batch.

6.3 Dashboard, reports and insights

View	What it shows (completed bills only)
Dashboard summary	today's sales total and bill count, product count, low-stock count, recent bills
Insights (last N days)	revenue, bills, average bill, items sold, customers and gross profit, each vs the previous period; daily revenue; weekday x hour heatmap; payment methods; categories; top products and customers; walk-in vs known customers
Daily sales report	totals per store-local day
Top products report	ranked by units or revenue for a date range

6.4 Development data tools

Development builds can seed sample data. All three tools are refused outside Development, and the data they create is marked as invented in the docs and the CLI output.

Tool	What it does
Database:SeedSampleData	16 sample products with stock for each newly registered shop
seed-demo-history --email --days	backfills invented bills before the shop's first real activity, keeping the stock ledger consistent; all or nothing
reset-demo-data --profile grocery|pharmacy --confirm	replaces one shop's products, customers and sales with a profile; touches only that ShopId; back up first

7 AI assistant

Owners ask questions about their shop in plain language, on the dashboard or the Assistant page. A local model (qwen3.5:9b on Ollama) answers by calling read-only tools that run the same queries as the reports. Every figure in an answer is checked against the data before the owner sees it.

7.1 Store tools

Tool	Arguments (defaults)	Returns
GetTodaySales / GetYesterdaySales	none	total paid, bills, average bill
GetTopSellingProducts	period, limit 1-20 (5), rankBy units|revenue	top products
GetMostProfitableProducts	period, limit 1-20 (5)	ranked by gross profit
GetLowStockProducts	limit 1-50 (20)	the Inventory page's Low filter
GetExpiringProducts	withinDays 1-365 (30), limit 1-50	expired or expiring soon
GetInventoryValue	none	stock x cost price and x selling price
GetUnsoldProducts	days 7-365 (30), limit 1-50	active products with no sale

Periods are named (today, yesterday, last7days, thisMonth, lastMonth, last30days) and turned into dates by the server in the store time zone. No tool takes a shop, user or date, and there is no free-form SQL.

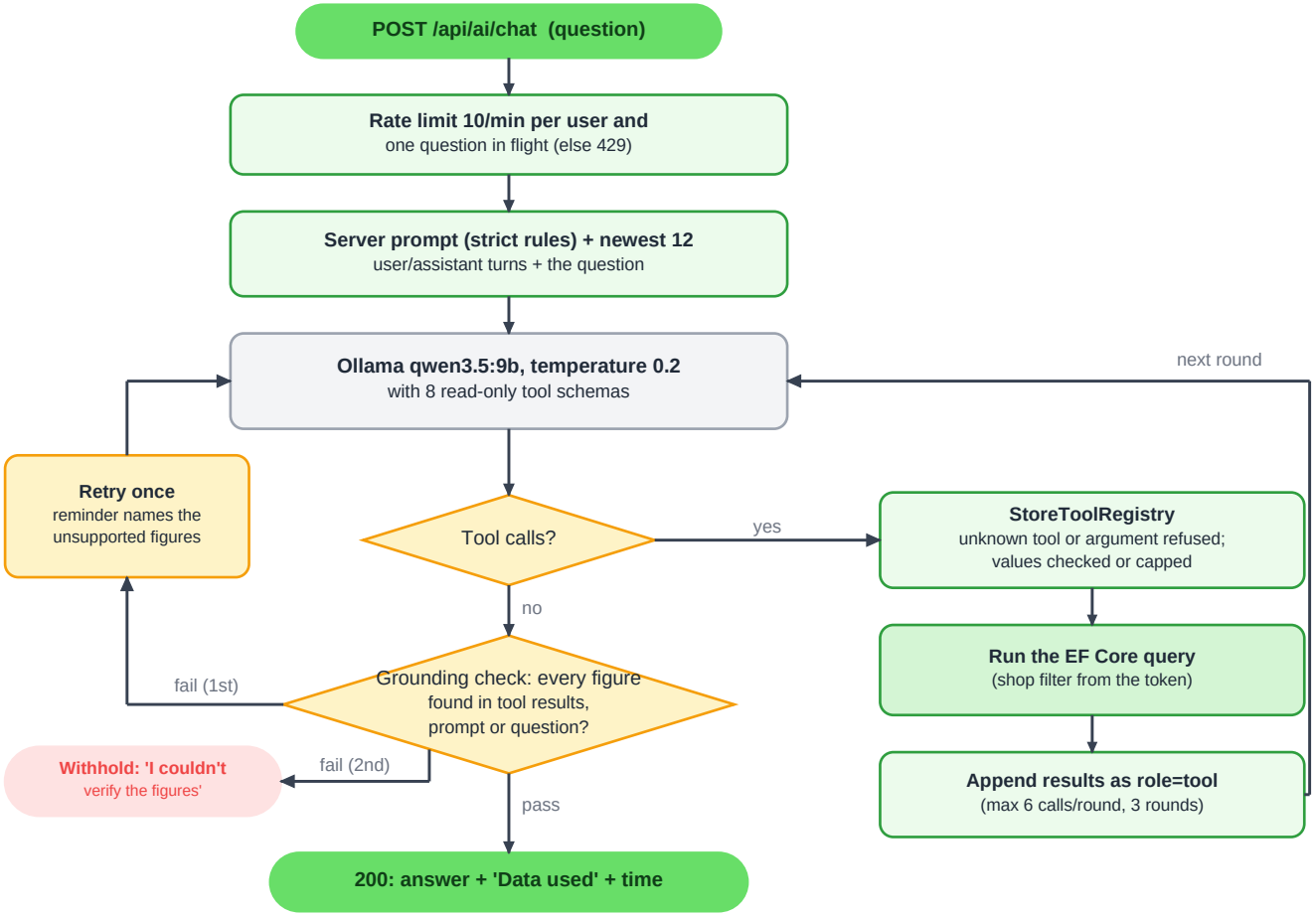


Figure 15. How a question is answered, including the grounding check.

7.2 The grounding check

Every number in the final answer must appear in this question's tool results, the server's prompt or the question. Formatting may differ (10,592.46, 10592.46, 1,23,456, or rounded to whole rupees); the value may not. Whole numbers up to 10 are always allowed. Numbers from the conversation history do not count, because the browser sends the history.

7.3 Why the strict prompt

Measured on the development Mac with a two-part question ('today's sales and low-stock products'), two runs per setting:

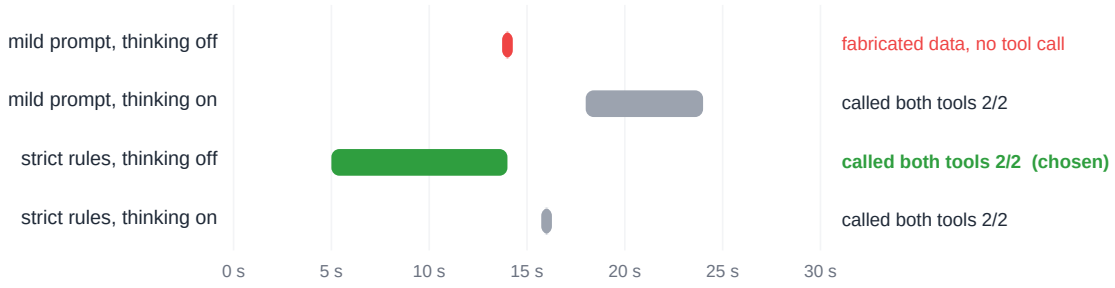


Figure 16. Time per model turn by prompt and thinking mode. The mild prompt invented figures.

- Answers are not streamed, on purpose: the grounding check needs the whole answer before anyone sees it.
- Message text and tool results are never logged; logs record the model, timings, token counts and tool names.
- Live test: 12 of 12 questions answered correctly when scored against independent SQL, 13-50 s each.

8 Demand forecasting (beta)

Each owner sees a 7-day demand forecast, stockout risk and reorder suggestions for their own shop, learned from the shop's recorded sales and stock. One model is trained on every shop's history (the shop is a feature), and an owner only ever sees their own shop's results.

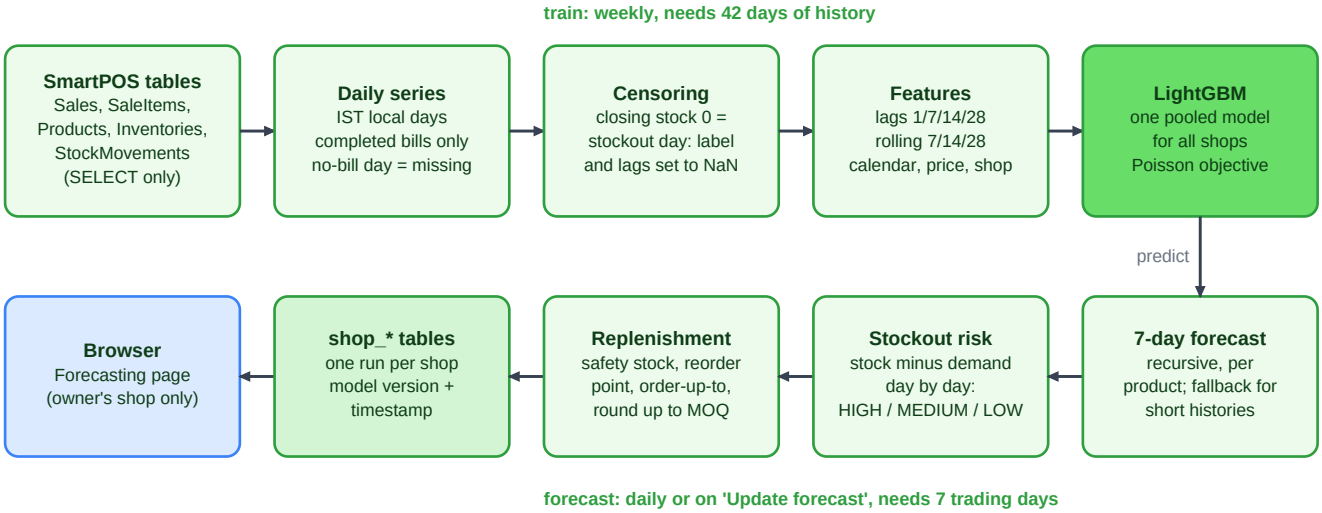


Figure 17. From SmartPOS tables to the owner's screen.

8.1 Turning sales into daily series

Question	Answer
What is a day?	The shop's local day (Asia/Kolkata). A bill at 00:30 IST counts on that day, though it is the previous day in UTC.
Units sold	Sum of SaleItems.Quantity on bills with Status = Completed.
Day with no bills at all	The shop was closed or didn't record sales: every product is **missing** that day, not zero. On a trading day, a product that didn't sell **is** a zero.
Stockout day	Closing stock = QuantityAfter of the day's last movement, carried forward. Ending at 0 means sales understate demand: the day is censored.
Price	Average selling price that day, carried forward; catalogue price if never sold.
Which products	Active products; inactive ones still count as history.
How much history	Up to yesterday. 7 trading days to forecast a shop; 42 days to train.

Data on 5 November 2026: 3 shops, 92 / 90 / 89 trading days (31 Jul to 3 Nov), 50 products, 7,794 completed bills, 5 / 1 / 1 stockout days.

8.2 Features

Group	Features
Lags	lag_1, lag_7, lag_14, lag_28
Rolling	rolling_mean_7 / 14 / 28, rolling_std_7 / 28
Calendar	day of week, day of month, month, week of year, weekend, holiday, festival name, days to festival
Product	product, category, unit price, discount, promotion flag, price change vs 28-day mean
Shop	store_id = the SmartPOS shop id (categorical)
History	revenue_rolling_28, censored_share_28 (stockout days in the window), series age

No leakage. Every lag and window for day d reads only days before d. A test overwrites every value on and after d with garbage and checks that no feature for d changes. One function computes features for training, backtests and each step of the forecast, so training and serving cannot drift apart.

8.3 Training

The label is observed sales on trading days that did not end out of stock. Data is split by time, never at random, so the model is always judged on days after the ones it learned from.

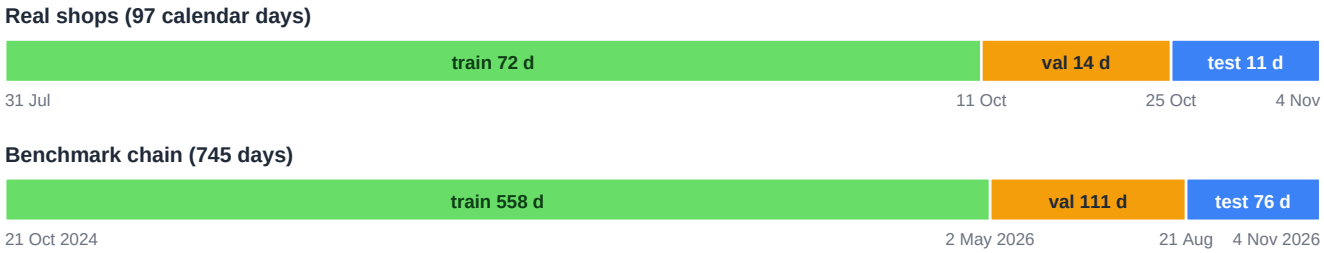


Figure 18. Chronological train / validation / test split for the real shops and the benchmark.

Step	Real shops	Benchmark chain
Candidates (validation WAPE)	Poisson 0.388 (chosen, 120 trees), Tweedie 1.2 0.390, squared error 0.390	Poisson 0.4091, Tweedie 1.2 0.4080 (chosen, 546 trees), squared error 0.4127
Tree size	15 leaves, 20 rows per leaf (small data)	63 leaves, 40 rows per leaf
Evaluation	refit on train + validation; test used once	same
Production model	same settings refit on all history: shops-lgbm-20261104-44474e72, about 1.5 s	lgbm-20261104-9591d4d6, 44 s, about 3.3 GB peak

Learning rate 0.05, early stopping on validation, deterministic. The artifact's SHA-256 is stored in model_metadata and checked before every use; the newest trained model becomes the active one for its source.

8.4 Accuracy on the real shops

WAPE = total absolute error / total sales; lower is better. Test period 25 Oct to 4 Nov, 243 product-days.

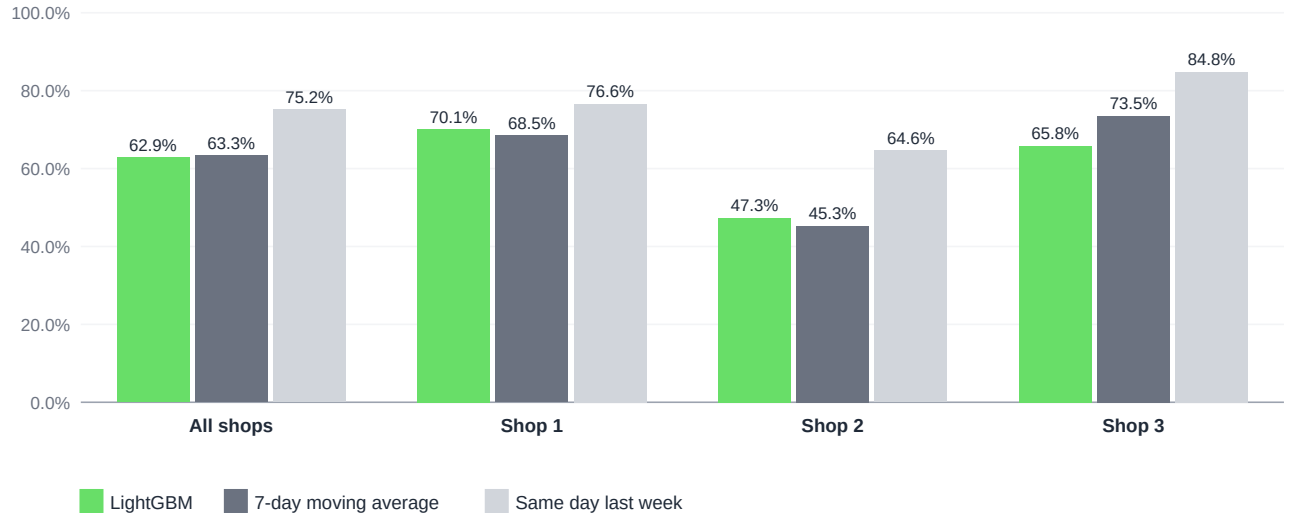


Figure 19. Test-period WAPE by shop.

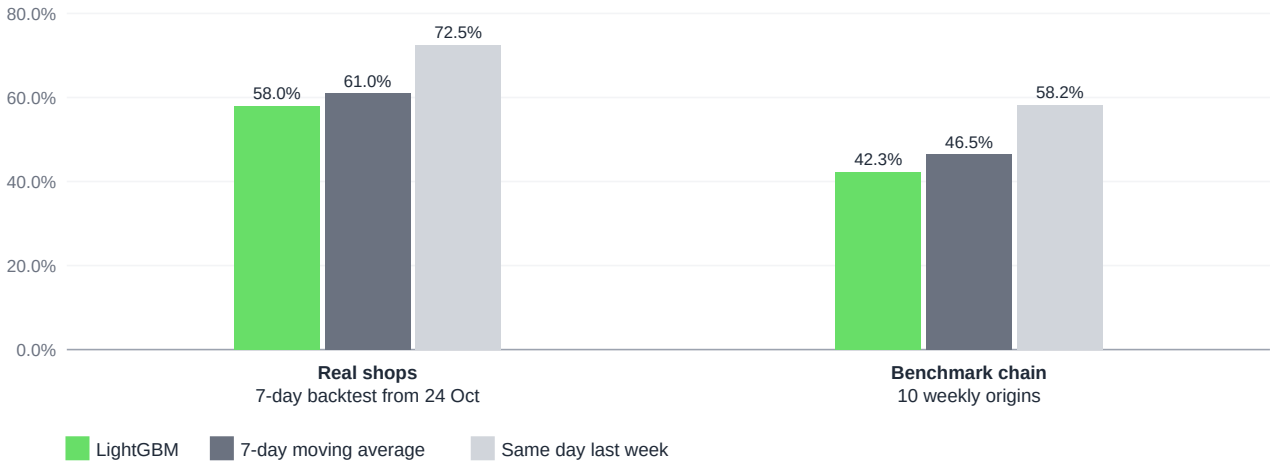


Figure 20. Full 7-day recursive backtests: what production can actually know.

Honest beta. With three months of history the model is level with a 7-day moving average overall: better for shop 3, slightly worse for shops 1 and 2, and forecasts run high (bias +13%). The value today is in turning a reasonable demand estimate plus each product's stock, lead time and minimum order into concrete stockout dates and order quantities. Weekly, seasonal and festival patterns need a year or more of history.

8.5 Making the 7-day forecast

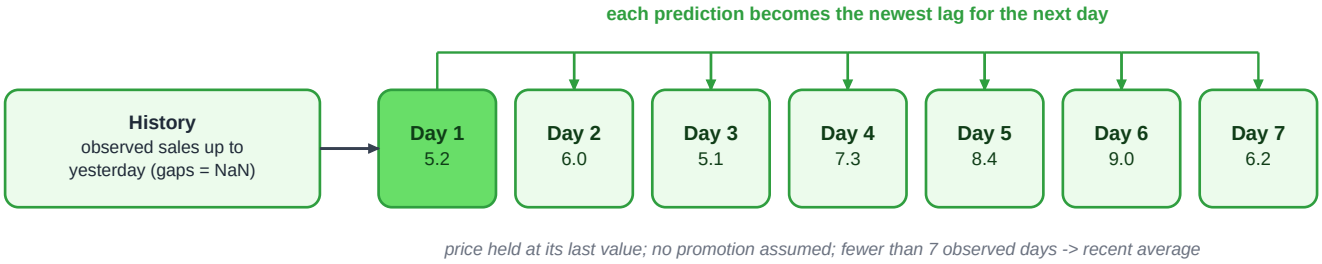


Figure 21. Recursive forecasting: day 1 feeds day 2, and so on.

Category forecasts are not modelled separately: they are the sums of the product forecasts of the same run, so they always add up (tested in the API and the web app). Each run is written in one transaction with its model version and timestamp.

8.6 Stockout risk

Stock on hand now (what the owner sees on the Inventory page) minus the forecast, day by day. The first day demand cannot be met is the predicted stockout date. SmartPOS has no purchase orders, so nothing is on order. Today counts in full, which is slightly cautious.

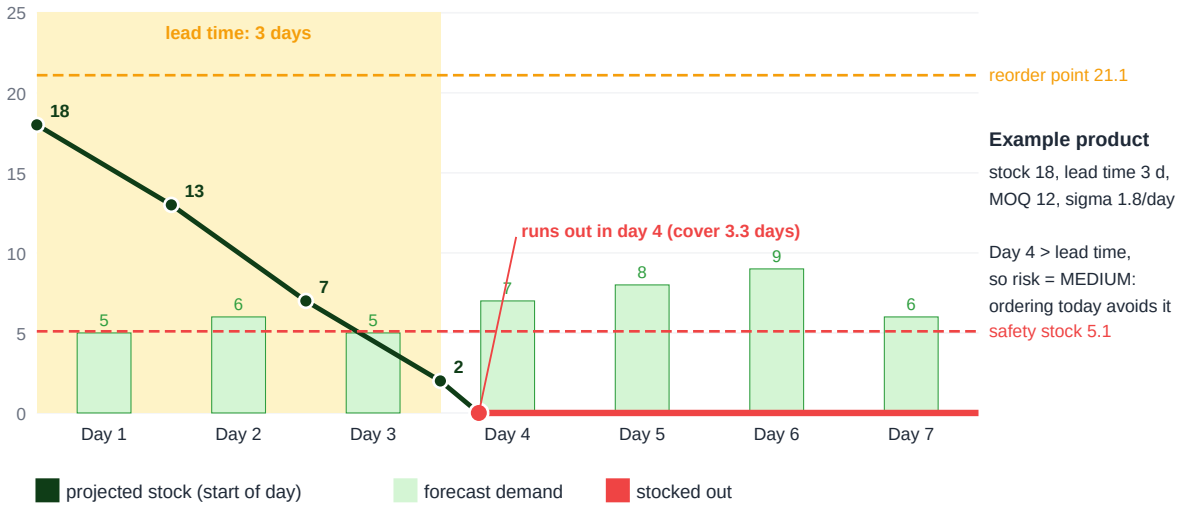


Figure 22. Worked example: projected stock against forecast demand.

Risk bands are transparent rules, not probabilities. They are applied in this order:

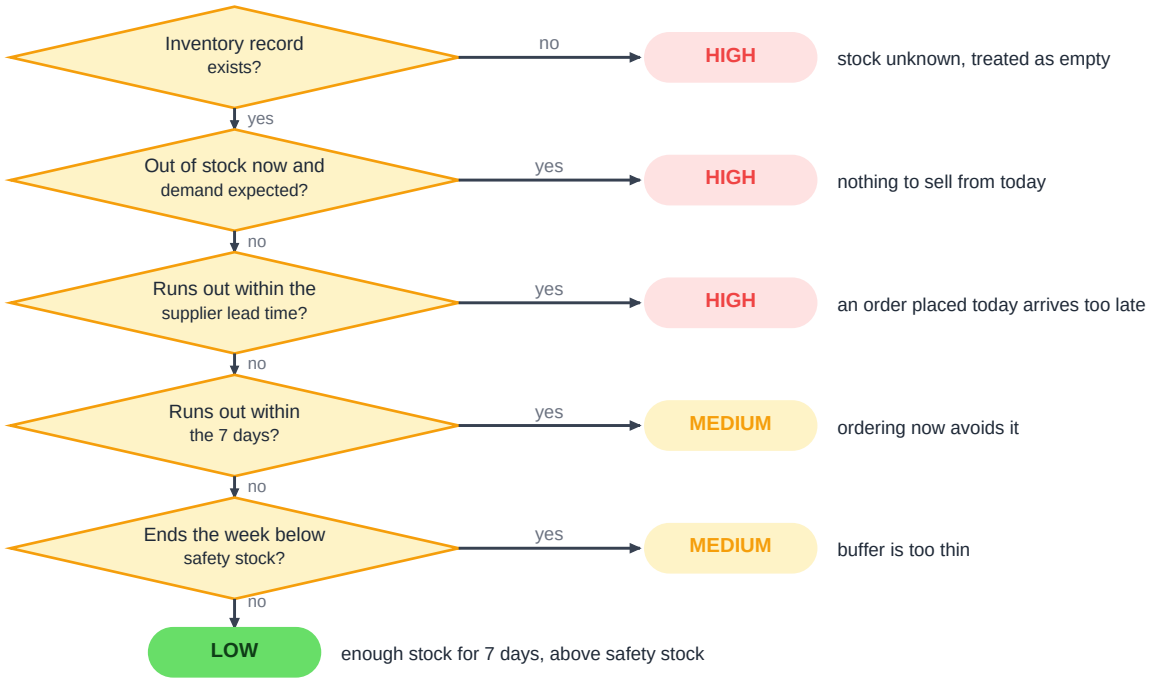


Figure 23. Risk band decision rules.

8.7 Reorder suggestions

A periodic-review, order-up-to policy. Safety stock covers demand variability during the lead time; an order is suggested once stock plus anything on order is at or below the reorder point, and rounded up to the supplier's case size. Every row carries a plain-language reason.

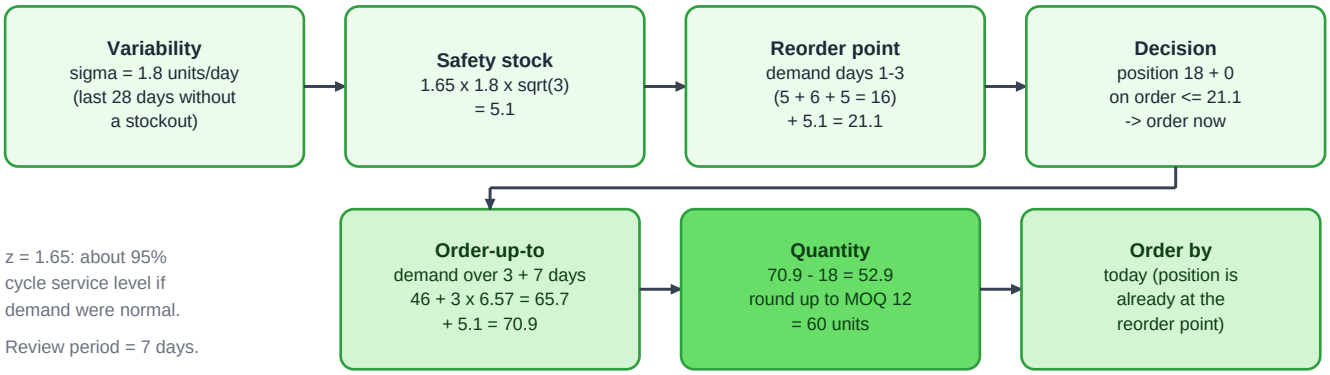


Figure 24. Worked example for the same product: order 60 units today.

8.8 Current runs

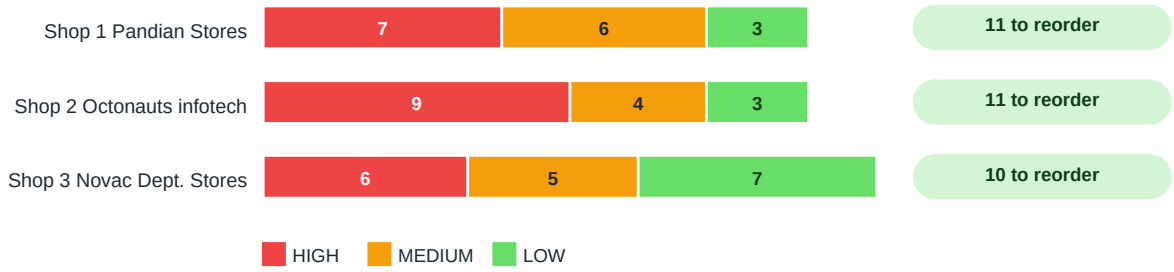


Figure 25. Risk mix per shop for 5-11 November 2026 (16, 16 and 18 products).

Many products are HIGH because no restocks have been recorded in three months, so stock has only gone down. Recording deliveries as inventory adjustments fixes both the alerts and the stockout-day detection.

8.9 The synthetic benchmark: why and how

Real shops can't tell us how much demand went unmet on stockout days, or how close a forecast is to the best possible one. A simulated 10-store Tamil Nadu chain with **known** demand can. It is used only offline, to check the method; it is never served by the API or shown in the app, and it has its own active model (data_source = synthetic).

Benchmark chain	
Stores	10: Chennai (3), Coimbatore (2), Madurai, Trichy, Salem, Erode, Tirunelveli; hypermarket, supermarket, neighbourhood
Products	105 in 7 categories: Grocery, Dairy, Beverages, Snacks, Personal Care, Household, Fruits and Vegetables
Suppliers	12, lead times 1-10 days, with delay probabilities
History	745 days, 21 Oct 2024 to 4 Nov 2026 (two Diwalis, two Pongals)
Rows	759,590 daily sales and inventory days; 112,824 purchase orders
Demand	6.66 M units wanted, 6.51 M sold, 154,617 lost to stockouts (2.32%)
Seed	20261105: the whole dataset is reproducible

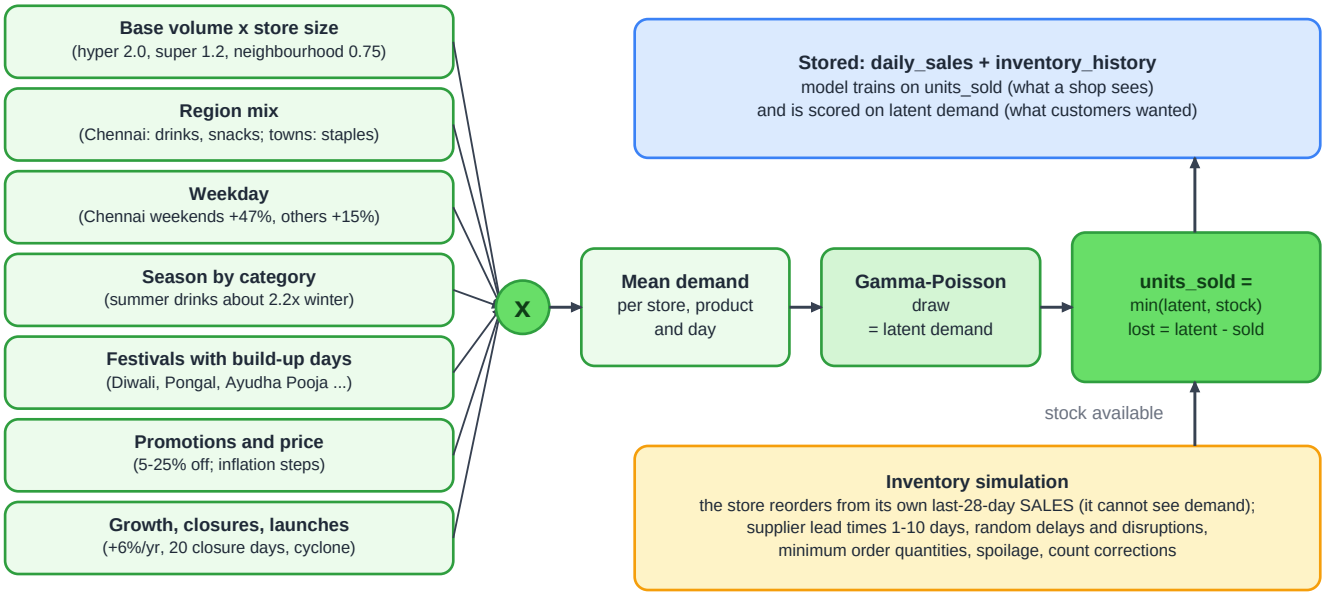


Figure 26. How the generator builds demand, stock and sales.

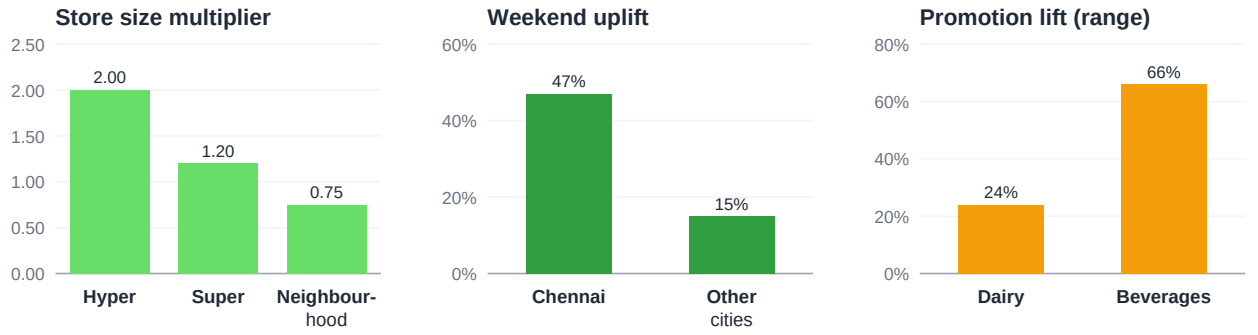


Figure 27. Patterns built into the generator and measured back from the loaded data.

- **Festivals** have build-up days and product-specific peaks: murukku x3.5 at Diwali; jaggery x3.5 and sugarcane and fresh turmeric x30 at Pongal; puffed rice x6 at Ayudha Pooja. Pongal is stronger outside Chennai; Christmas and New Year stronger in Chennai.
- **Stock** runs the store's own reorder policy from its recent sales, so stockouts happen the way they do in real shops: mostly on spike days.
- **Data quality**: 14 SQL checks (duplicates, negatives, sold > wanted, ledger balance, orphans, order dates) all return 0 on the loaded data.

8.10 Benchmark results

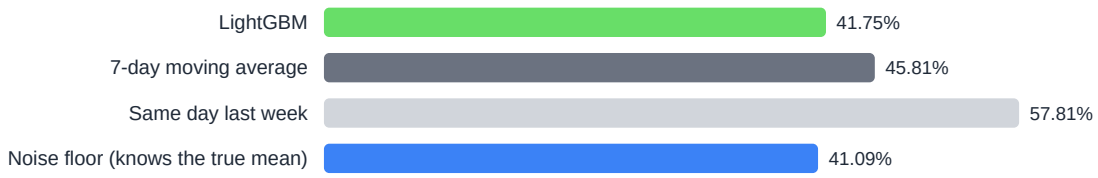
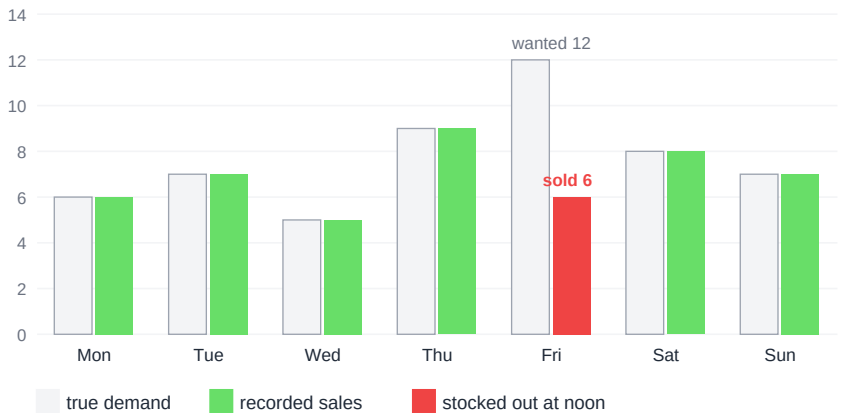


Figure 28. Day-ahead test WAPE against true demand, 78,910 store-product-days.

Single store-product-days are very noisy: even a forecaster that knew each day's true mean would score 41.09%. LightGBM reaches 41.75%, closing 86% of the gap between the moving average and that floor, and wins in every store (0.38 hypermarkets to 0.47 neighbourhood stores) and every category.

8.11 Why stockout days are left out



What the model learns from Friday

Naive: label = 6
teaches the model that
busy Fridays sell 6

Censoring-aware: label = NaN
day left out of training
and of the lag features

Illustrative numbers.

Figure 29. A Friday that sold out at noon: recorded sales understate demand.



Figure 30. Benchmark: bias on the 1,846 stockout days of the test period (same model, two ways of training).

Even when trained correctly, stockout days are under-forecast: they cluster on festival build-ups, which are also the hardest days to predict.

8.12 Operations and known limits

- **Daily**: *python -m novac_forecast forecast* after closing, or the owner presses Update forecast (3 per minute per shop).
- **Weekly**: *train*, then *forecast*. Watch WAPE and bias per shop in model_evaluation_metrics.
- A new shop needs 7 trading days; until the next retrain the model has not seen its id and relies on product and category history.
- No promotion calendar or purchase orders yet; a product that sold out and was restocked the same day is not flagged; risk bands are rules, not probabilities.

9 API reference

Base path /api. JSON in and out; errors are RFC 7807 problem+json with a title and a readable detail. All routes need a Bearer token except login, register and verify-email.

Method	Path	Purpose
POST	/auth/register, /auth/login	create an account + shop / sign in; both return a JWT
POST	/auth/verify-email, /auth/resend-verification	confirm the email link / send a new one
GET	/auth/me	the signed-in user
GET POST	/products	list (search, category, active, paging) / create with opening stock
GET PUT DELETE	/products/{id}	read / update (no stock field) / delete if never sold
GET	/categories	category names in use
GET	/inventory, /inventory/movements	stock with status filters / the ledger
POST	/inventory/adjustments	change stock with a reason
GET POST PUT	/customers, /customers/{id}	list, create, read, update
POST GET	/sales, /sales/{id}	complete a sale / list (from, to, search) / bill detail
GET	/dashboard/summary, /dashboard/insights?days=	today / analytics
GET	/reports/daily-sales, /reports/top-products	reports
GET	/settings/pos	till settings (max discount, currency, loyalty)
POST GET	/ai/chat, /ai/status	ask a question / is the model ready
GET	/forecasting/status | forecast | categories | stockout-alerts | replenishment | products/{id}/forecast | metrics	the owner's shop's forecasts
POST	/forecasting/generate	new forecast run (409 under 7 trading days)
GET	/health (no /api)	database, ai-model and forecasting checks

10 Testing

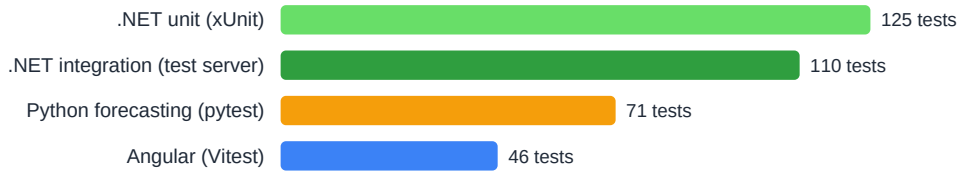


Figure 31. Automated tests per suite, all passing on 5 November 2026.

Suite	Covers
.NET unit	SaleCalculator (rounding, discount split, settle), StockRules, DiscountPolicy, loyalty, store calendar, search patterns, grounding check, tool argument validation
.NET integration	the full HTTP pipeline: register/login, products, sales (oversell 409, discount 403), inventory, customers, dashboard, reports, AI chat, every forecasting route with two owners reaching only their own shops
Python	SmartPOS reader on a throwaway schema with known answers (completed bills only, IST boundary, closed days missing, censoring, SmartPOS tables never written), training, runs, API isolation, migrations, features (leakage test), inventory rules, benchmark generator
Angular	cart arithmetic mirroring SaleCalculator, forecast view helpers (filters, totals, risk counts)
Against PostgreSQL	70-check smoke test including two-shop isolation and the 12-way oversell race
In a browser	Playwright walkthroughs: sign up, add a product, adjust stock, sell, dashboard; assistant; forecasting as two different owners, at desktop and phone width

Principle. Tests check behaviour against independently known answers (hand-built fixtures, independent SQL), not against the code's own output.